

Interrupt Controller

Programmable priority and preemption

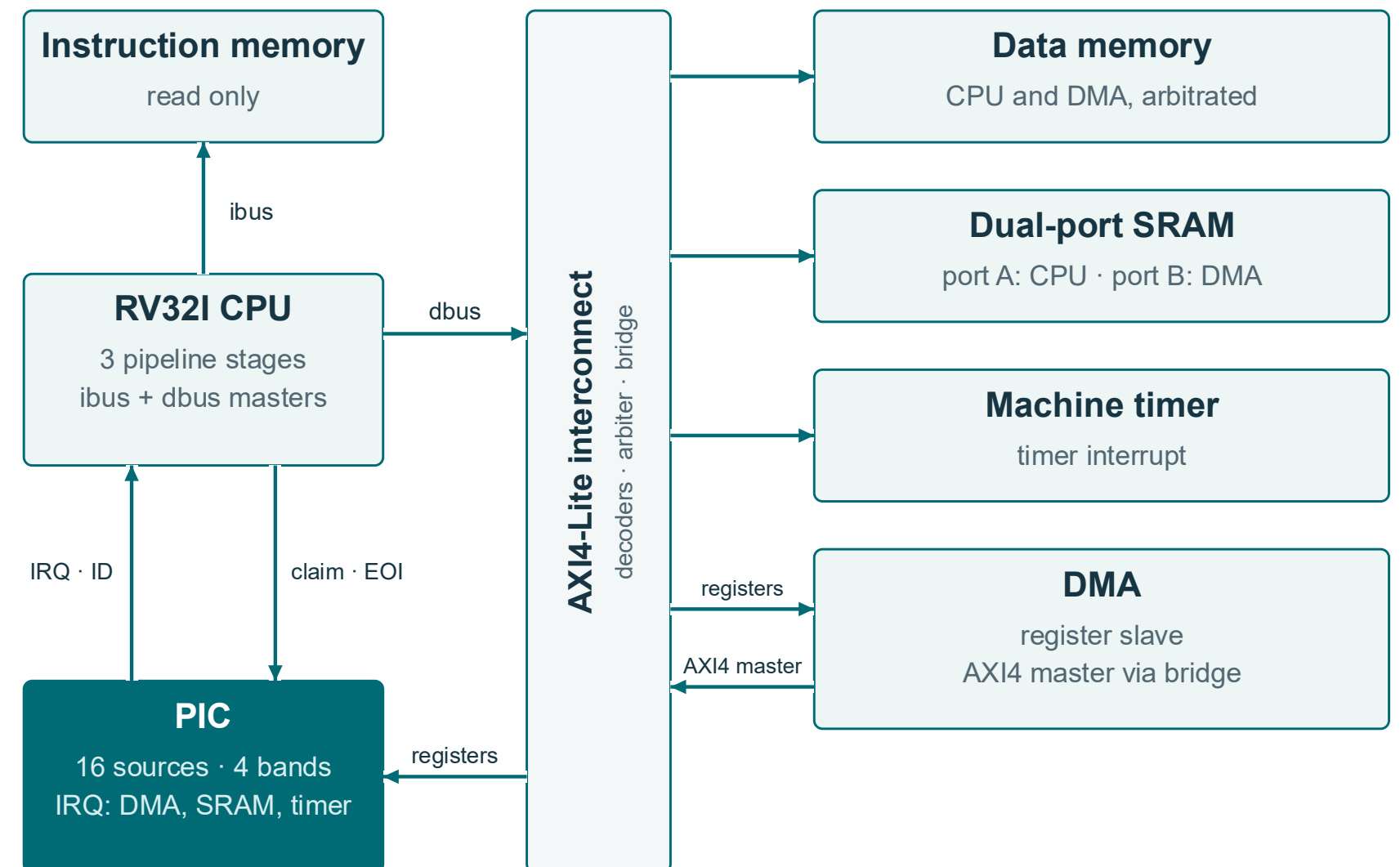
Bunea Cosmin-Andrei

Computer Engineering, year III

Academic year 2025–2026

Transilvania University of Braşov

RTL implementation · interfaces · verification



The task: a PIC with priority bands, nesting and deadlines

01 16 hardware sources + 16 software triggers

02 Level or edge mode for every source

03 Priority bands, not a flat priority

04 Preemption with nesting

05 Spurious interrupt detection

06 Deadline-based priority escalation

07 Software-triggered interrupts

08 AXI4-Lite register map, 11 registers

09 One request + vector to the CPU

10 SW interrupts identical to HW once they enter arbitration

The 32 HW/SW inputs merge into 16 source identities.

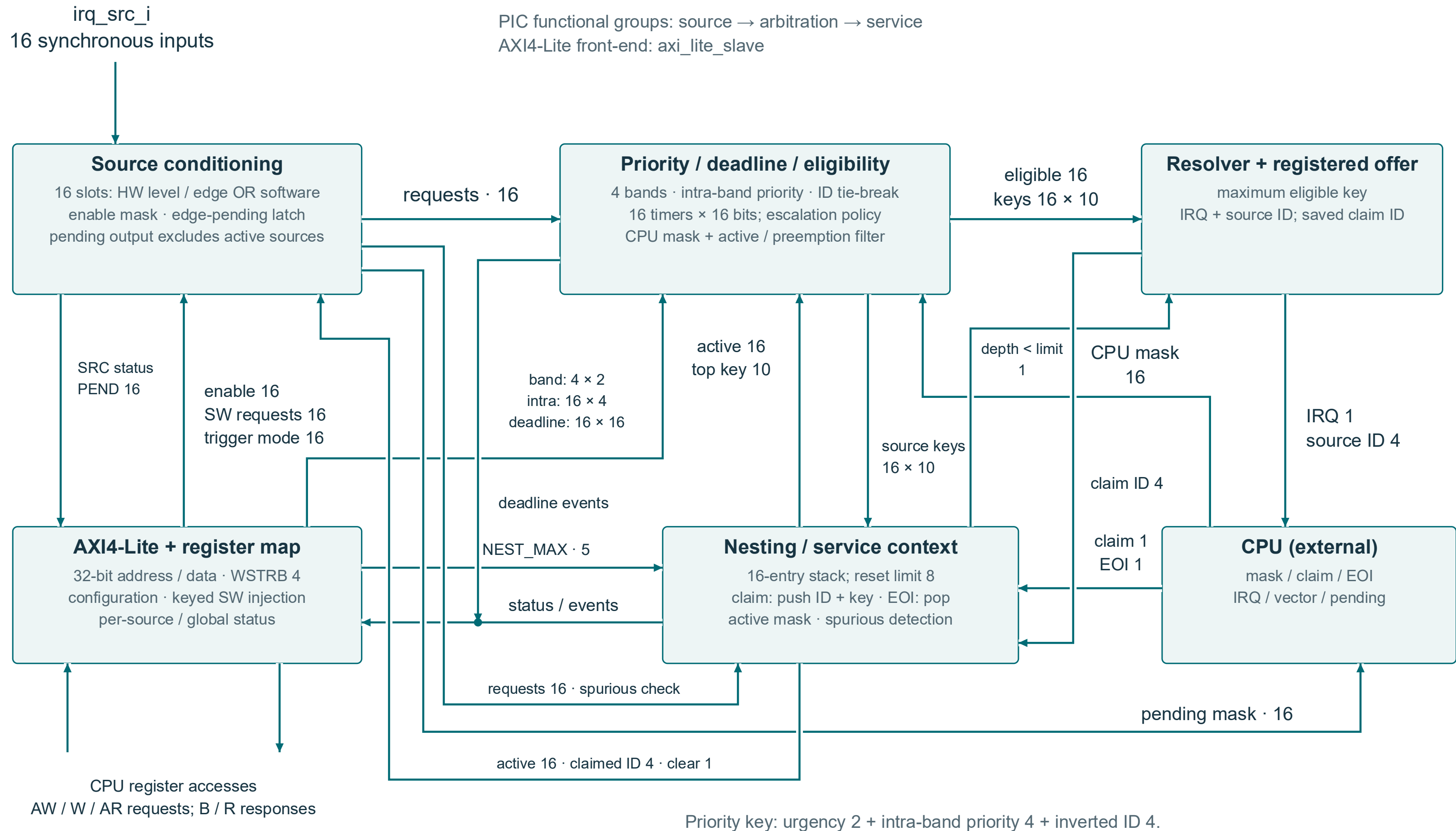
The brief left 21 questions open; five of them shaped the design

Question from the brief

What I decided

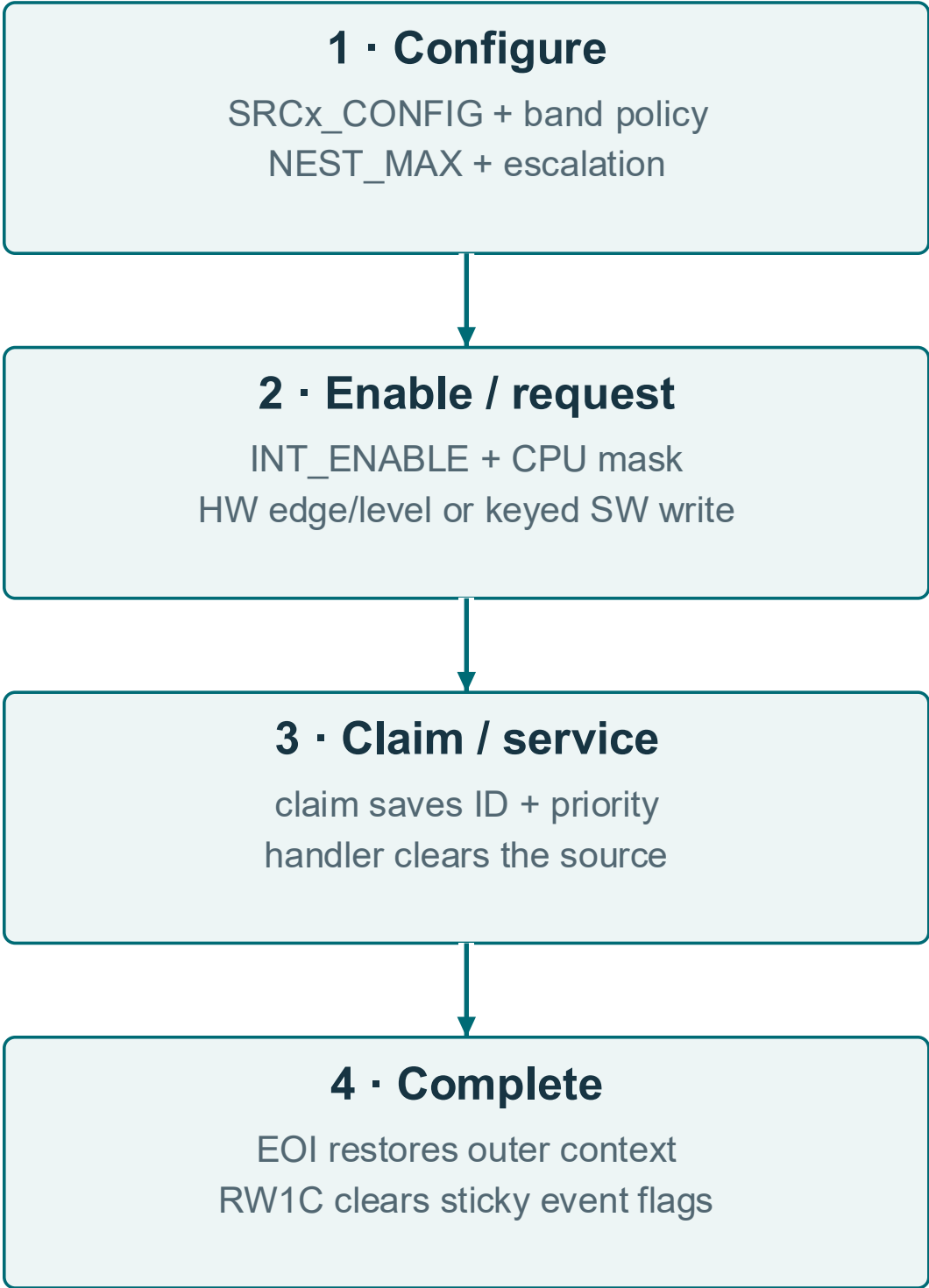
- | | | | |
|----------|--|---|---|
| 1 | How many bands, and how is a tie resolved? | → | 4 bands with programmable urgency; one 10-bit key per source; the lower ID wins a tie |
| 2 | What if an active source is moved to another band? | → | The key saved at claim stays; the new band applies to the next request |
| 3 | What happens when the nesting limit is reached? | → | No new offer and the overflow flag is set; active contexts are kept |
| 4 | When is a request spurious, and is it visible? | → | Checked at claim; the context is kept and closed by EOI; logged per source and globally |
| 5 | What does escalation mean? | → | Jump to a target band or bump one urgency step; may repeat; cleared when served |

A request becomes an active context only at claim



The registers set the policy and make the state observable

Address	Register / group	Role in use
0x00 + 4n	SRCn_CONFIG	Trigger, band, priority, deadline
0x40 + 4n	SRCn_SW_TRIG	Set with key 0xA5A5 in both upper byte lanes
0x80 + 4n	SRCn_STATUS	Pending, active, escalated, spurious
0xC0 / D4	BAND_CONFIG / ESCALATION_CFG	Band urgencies and escalation policy
0xC4 / C8 / CC	NEST_STATUS / NEST_MAX / ACTIVE_VEC	Active context and nesting limit
0xD8	INT_ENABLE	Per-source enable
0xD0 / DC	SPURIOUS_LOG / INT_STATUS	Logged events; cleared by RW1C

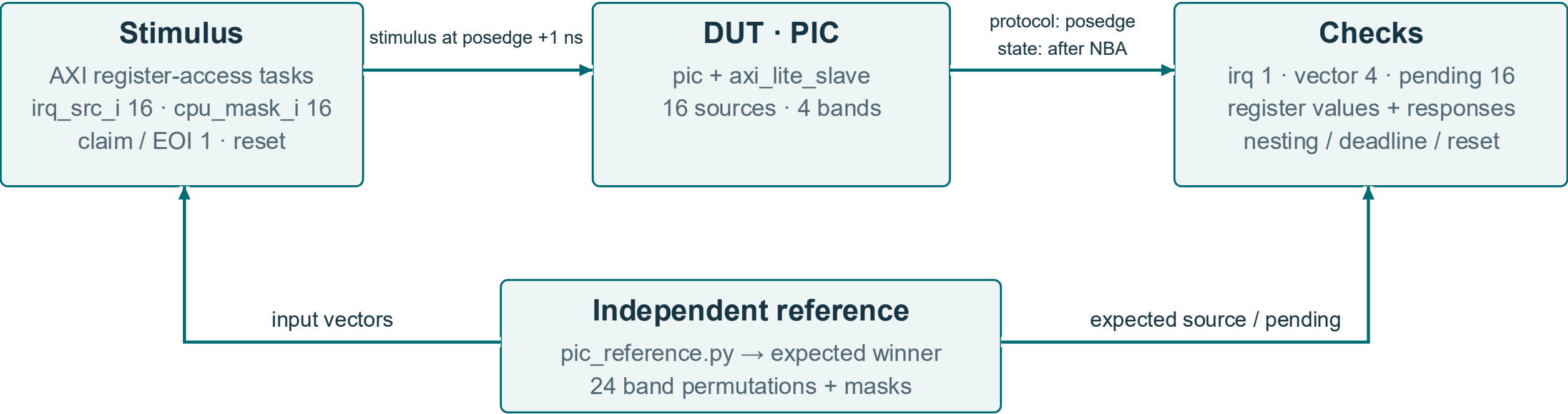


Register configuration → service sequence

The priority rules are checked against an independent model

Directed scenarios plus an independent priority model

Protocol and PIC invariants are checked by assertions attached through bind.



Independent model

All 24 band orders, with masks and ties

Same-cycle events

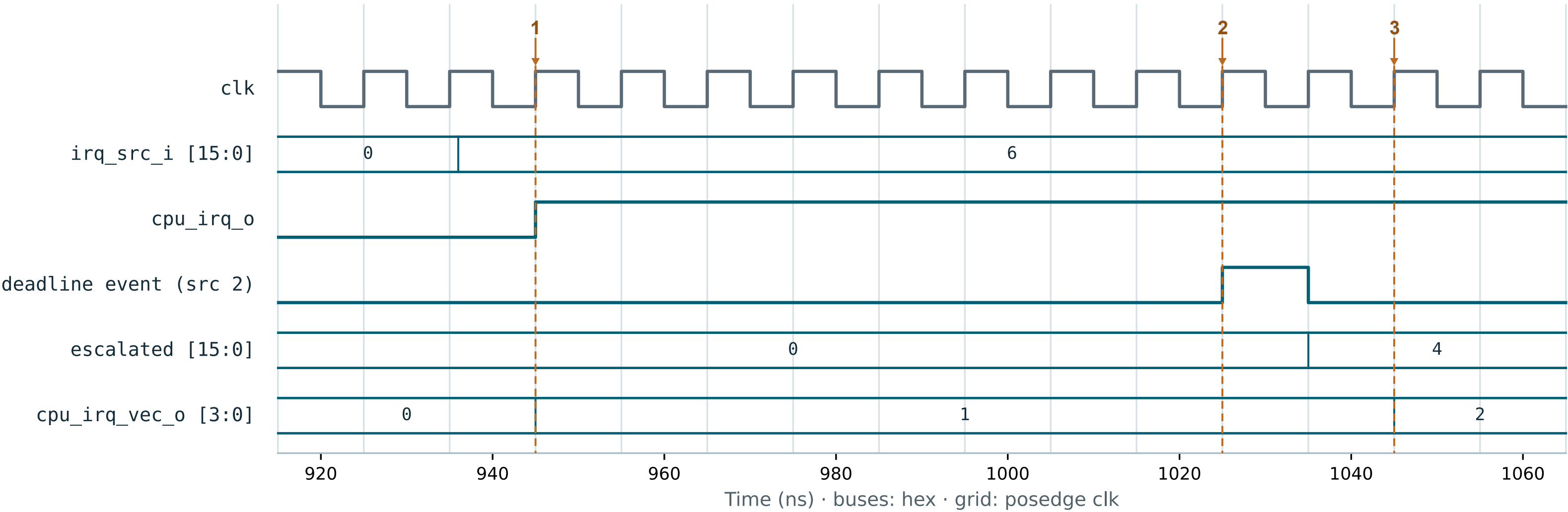
Edge at claim, masked source, strobed key

External AXI reference

Front-end compared with PULP: 96 cases

ModelSim: 22 CPU/PIC configurations · Verilator: 16 benches with assertions bound to the RTL

Escalation follows the programmed urgency, not the band number



1 · Source 1 offered
Same band: the lower ID wins the tie

2 · Source 2 deadline
10 cycles pending → escalation

3 · Offer moves to 2
Bumped to band 3, the next urgency

Bands reordered: band 0 is the least urgent. Bus values are hexadecimal.

Result: configurable arbitration with verified preemption

WHAT I BUILT

1. Banded priority with programmable urgency
2. Nesting stack, spurious handling, deadline escalation
3. Priority model covering all 24 band orders

WHAT I LEARNED

- Ties and same-cycle events need a written rule.
- Saving the key at claim keeps the stack valid after reconfiguration.
- The CPU mask belongs inside the PIC, or a masked source holds back the others.

Internship: brief → specification → RTL → verification, with a review at each step.